Ders 9 Genişletilmiş Notu: Kod Kusurları ve Refactoring Eylemleri (Duplicated Code, Long Method, Long Parameter List)

Ders 9 Genişletilmiş Notu: Kod Kusurları ve Refactoring Eylemleri (Duplicated Code, Long Method, Long Parameter List)

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Kod Kusurlarına Giriş ve Anlatım Biçimi
- 2. Duplicated Code: Yinelenen Kod
- 3. Extract Method: Metot Çıkartma
- 4. Long Method: Aşırı Uzun Kod
- 5. Yinelenen Kodu Toplamaya Yönelik Diğer Düzenlemeler
- 6. Pull Up Method: Metodu Yukarı Çekme
- 7. Aşırı Uzun Kodu Kısaltmaya Yönelik Diğer Düzenlemeler
- 8. Decompose Conditional: Karar Verme Komutunu Parçalama
- 9. Long Parameter List: Uzun Parametre Listesi
- 10. Introduce Parameter Object: Parametre Nesnesi Oluşturma
- 11. Preserve Whole Object: Tüm Nesneyi Gönder
- 12. Replace Temp with Query: Geçici Değişken Yerine Erişim Metodu
- 13. Replace Method with Method Object: Metot Yerine Yeni Türden Nesne
- Örnek Sorular
- Dersin Toparlaması

1. Kod Kusurlarına Giriş ve Anlatım Biçimi

Önceki derste Fowler'ın video kiralama örneğinde kullanılacak uygulama ve birim sınaması (JUnit) incelenmişti. Bu dersten itibaren tek tek kod kusurları (code smells) ve bunları gideren refactoring eylemleri tanıtılmaya başlanır.

Anlatım şu sırayla ilerler (00:07:25): önce bir kod kusuru görülür; tanımı, nasıl ve neden ortaya çıktığı, ne gibi sorunlara yol açabileceği söylenir. Ardından o kusuru giderebilecek refactoring eylemleri üzerinden eylemin ne olduğu anlatılır. Kusurlar ve eylemler arasındaki ilişki bire bir değildir: bir kusuru birkaç eylem giderebilir, bir eylem de birkaç kusura iyi gelir. Ders, en uygun eşleştirmeler üzerinden ilerler.

İçerik Martin Fowler'ın Refactoring kitabından alınmıştır. Kitaptaki kod kusurlarının hepsi değil, yaklaşık üçte biri derse dahil edilmiştir. Hoca bu kısmı bir giriş (introduction) olarak düşünmeyi, konu ilgi çekiyorsa gerisini kitaptan çalışmayı öneriyor. Bunu bir benzetmeyle anlattı: poşetten bir tutam ıhlamur çekmek isterken dalları yüzünden çok daha fazlası gelir; bir haftalık ek malzeme anlatayım derken üç haftalık malzeme çıkabilir.

Hoca bir kez daha hatırlattı (00:09:20): kod kusuru bir hata değildir. Kullanılan programlama yaklaşımının önerdiği biçimin dışındaki kullanım örnekleridir. Yazılımın esnekliğini düşüren

olgulardır; başka bir deyişle düşük olması gereken bağlaşımı yükselten, yüksek olması gereken uyumu düşüren olgulardır.

> **Hocanın vurgusu:** Kod kusurlarının ve refactoring eylemlerinin literatürdeki İngilizce adları bilinmelidir (00:09:55). Slaytlarda İngilizce ad başlıkta, uygun Türkçe karşılık altında yer alır.

2. Duplicated Code: Yinelenen Kod

İlk kod kusuru Duplicated Code'dur (ekrandan, 00:10:00). Slayttaki tanım:

- Aynı kodun veya çok benzer kod yapısının birden fazla yerde tekrarlanması durumudur.
- Bu kodun işlevselliğinde değişiklik yapılması gerektiğinde, kodun tekrarlandığı her yerde aynı işlemlerin tekrarlanması gerekecektir.
- Ortaya çıkan sonuç bir bakım kabusudur.
- Çözüme götürecek düzenlemeler: Extract Method (yinelenen kodu bir metot altında toplamak amacıyla kullanılır); yinelenen kodu toplamaya yönelik diğer düzenlemelere ileride değinilecektir.

Hoca bu kusurun genelde kopyala-yapıştır programlamanın sonucu olduğunu söyledi. Kod kopyalanıp yapıştırılır, yapıştırılan yerdeki duruma göre küçük düzenlemeler yapılır. Tanım "çok benzer kod yapısını" da kapsadığı için bu küçük farklar kodu kusurdan kurtarmaz.

2.1 Üye alan adlandırma

Aynı slaytta bir adlandırma notu vardır (ekrandan, 00:10:00): kod örneklerinde alt çizgi ile başlayan değişkenler (`_name`, `_orders` gibi) nesnenin üye alanlarıdır; IDE'lerde ise üye alanlar farklı renkle vurgulanır. Hoca önceki derste de söylediği gibi bu alışkanlığın, refactoring öncesi ve sonrası kod siyah-beyaz tek renk gösterildiğinde neyin üye alan, neyin parametre olduğunu ayırt etmek için kullanıldığını hatırlattı. IDE ile çalışırken buna gerek yoktur.

3. Extract Method: Metot Çıkartma

3.1 Tanım

Hoca belirli bir kusuru gideren belirli bir eylem biçiminde ilerlediği için kusuru ayrıca örneklemeye çoğu zaman gerek olmadığını, kusurun eylemin örneğinde zaten görüleceğini söyledi. Tanım kalıcı olmayacak kadar soyut kaldığında kusur ayrıca kod üzerinde gösterilecektir.

Slayttaki tanım (ekrandan, 00:14:20):

- Bir metot içindeki kodun bir kısmını yeni bir metot olarak belirlemeye dayanır.
- Bu yapılandırma birçok kod kusurunu (smell) düzeltir.
- Oluşturulan yeni metoda, amacını açıkça belirten, isabetli bir ad verilmelidir. Böylece alt düzey ayrıntılarla uğraşmadan kodun anlaşılabilirliği artar.

Extract Method yalnız Duplicated Code için değil, aşırı uzun metotları toparlamak için de kullanılır. Hoca adlandırmanın önemini vurguladı: metot çıkarıldıktan sonra ona amacını açıkça belirten bir ad vermek, kodun alt düzey ayrıntılara girmeden anlaşılmasını sağlar.

3.2 İlk örnek: printOwing ve printDetails

Slaytlarda refactoring'den önceki hali solda, sonraki hali sağda gösterilir (ekrandan, 00:14:20):

```
/*** ÖNCE: ***/
void printOwing(double amount) {
    printBanner();
    //print details
    System.out.println("name:" + _name);
    System.out.println("amount" + amount);
}
```

```
/*** SONRA: ***/
void printOwing(double amount) {
    printBanner();
    printDetails(amount);
}
void printDetails (double amount) {
    System.out.println("name:" + _name);
    System.out.println("amount" + amount);
}
```

Kod parçadır, derlenmedi.

`printOwing` metodu borcu ekrana yazdırır. Önceki halde önce `printBanner()` çağrılır (bir başlık yazılır), sonra programcının "print details" yorumuyla işaretlediği ayrıntılar (ad ve miktar) yazılır. Yani bu metotta iki ayrı iş yapılmaktadır. Ayrıntıların yazılması kesilip `printDetails` adlı yeni bir metoda çıkarılır.

Burada dikkat edilecek nokta parametredir: `amount` özgün metodun parametresidir ve ayrıntılar kısmında kullanılır. Bu kısım yeni metoda taşındığı için `amount` yeni metotta da parametre olarak tanımlanmalı ve özgün metottan yeni metot bu parametreyle çağrılmalıdır.

Sonraki halde yorum satırı ortadan kalkmıştır. Metot adları iyi seçildiği için `printOwing`'e bakan kişi önce başlığın, sonra ayrıntıların yazıldığını hemen anlar. Başlık yazdırmada bir sorun ya da değişiklik isteği olursa yalnız `printBanner`'a, ayrıntılarda bir sorun olursa yalnız `printDetails`'e gidilir. Uzun bir metodun içinde "nerede değiştirmeliyim, hata nerede" diye aramak yerine sorun ikiye bölünmüş olur.

Hoca bunu yemek benzetmesiyle anlattı: büyük bir lokmayı yutmak zordur, küçük lokmalar daha kolay ve hızlı yutulur. "Bir seferde her şeyi birden yapayım" yaklaşımı her zaman doğru değildir.

Soru: (hocanın kendi sorduğu, 00:17:51) Önceki halde topu topu üç komut vardı; şimdi iki metot oldu ve kod toplamda biraz uzadı. Bu bir kazanç mı? **Cevap:** Bu kadar basit bir örnekte itiraz haklıdır. Örnek yalnız eylemin nasıl yapıldığını göstermek içindir. Eylem asıl daha uzun metotlarda kazandırır.

> **Hocanın vurgusu:** Programlamaya yeni başlayanlara her yere yorum yazmaları söylenir. Buradaki hedef farklıdır: hiçbir yorum satırına gerek kalmadan programa bakanın onu anlayabileceği biçimde, anlamlı adlarla kod yazmak (00:18:45).

4. Long Method: Aşırı Uzun Kod

4.1 Tanım

İkinci kod kusuru Long Method'dur; hoca vurgulamak için Türkçesini "aşırı uzun kod" diye çeviriyor (ekrandan, 00:20:00). Slayttaki tanım:

- Bir metodun uzunluğu arttıkça anlaşılması zorlaşır, değişikliklerden etkilenme olasılığı artar, birden fazla ilgi alanını kapsama olasılığı artar (uyumun düşmesi).
- Çözüme götürecek düzenlemeler: Extract Method (uyumlu, yani belli bir ortak amaca yönelik komutlar seçilerek bir metot altında toplanır); aşırı uzun kodu kısaltmaya yönelik diğer düzenlemelere ileride değinilecektir.

Hoca maddeleri şöyle açtı. Metot uzadıkça anlaşılması zorlaşır. Uzun metot, hem hatalar hem de gereksinim değişiklikleri için daha büyük bir hedef olur. Ne kadar uzunsa o kadar çok iş yapıyor demektir ve bu işlerin birbiriyle ilgisinin azalma riski artar; bu da uyumu düşürür. Bağlaşım ve uyum en temel iki iç kalite ölçütüdür. Yani bu olgular yanlış çalışma değil, kodun kalitesini düşüren olgulardır.

4.2 İkinci Extract Method örneği

Hoca daha anlaşılır bulduğu ikinci örneği gösterdi (ekrandan, 00:21:30):

```
/*** ÖNCE: ***/
void printOwing( ) {
```

```
  Enumeration e = _orders.elements();
  double outstanding = 0.0;

  // print banner
  System.out.println("*****************");
  System.out.println("* Customer Owes *");
  System.out.println("*****************");

  // calculate outstanding
  while (e.hasMoreElements()) {
    Order each = (Order) e.nextElement();
    outstanding += each.getAmount();
  }

  //print details
  System.out.println ("name:" + _name);
  System.out.println ("amount" +
    outstanding);
}

/*** SONRA: ***/
void printOwing( ) {
  printBanner();
  printDetails(getOutstanding());
}

void printBanner() {
  System.out.println("*****************");
  System.out.println("* Customer Owes *");
  System.out.println("*****************");
}

double getOutstanding() {
  double outstanding = 0.0;
  for (Order each : _orders)
    outstanding += each.getAmount();
  /* for-each ile döngü kısaldı! */
  return outstanding;
}

void printDetails (double outstanding) {
  System.out.println ("name:" + _name);
  System.out.println ("amount" +
    outstanding);
}
```

Kod parçadır, derlenmedi.

Önceki halde metot borç makbuzunu yazar. İki geçici değişken vardır: biri nesne (`Enumeration e`), biri ilkel tür (`double outstanding`). Ardından yorumlarla işaretlenmiş üç iş gelir: banner yazdırma, döngüde her sipariş için tutarı alıp birikimli toplam olarak `outstanding`'e ekleme (hoca bunu video örneğindeki `Rental` gibi düşünmeyi önerdi) ve ad ile borcun yazdırılması. Hoca yorumları saymadan metodun yaklaşık 10 satır olduğunu saydı.

Metodun üç ayrı iş yaptığı yorumlardan da görüldüğü için:

1. Ayrıntıların yazılması `printDetails` metoduna çıkarılır; önceki örnekteki gibi borç (`outstanding`) ona parametre olarak verilir.
2. Döngüdeki birikimli toplam hesabı başlı başına bir iştir; `getOutstanding` adlı metoda çıkarılır. `outstanding` geçici değişkenine artık yalnız bu metotta ihtiyaç vardır. Ayrıca `Enumeration` ile `while` döngüsü yerine kısa olduğu için for-each döngüsü kullanılır; metot sonucu döndürür.

3. Banner yazılması da ayrı bir iştir; `printBanner` metoduna çıkarılır.

Sonuçta `printOwing` önce `printBanner()`'ı çağırır, sonra ayrıntıları yazar. Ayrıntılar yazılırken toplam borcun hesaplanıp verilmesi gerekir; bu da `getOutstanding()` çağrısıyla yapılır. Kod daha odaklanmış metotlar içerir hale gelir ve yorumlara gerek kalmaz.

Hoca toplam satır sayısının artabileceğini ya da aynı kalabileceğini, metot sayısının arttığını kabul etti. Önemli olan metotların kısalmasıdır: her biri küçük bir lokmadır, gerekirse hızla okunup anlaşılır ve düzeltilir. Ayrıca çıkarılan metotlar başka yerlerde de işe yarayabilir; orada yeniden kod yazmak yerine metot çağrılır ve yinelenen kod daha ortaya çıkmadan engellenmiş olur.

4.3 Metot sayısının artması ve çağrı yükü

Soru: (hocanın öğrencilerin aklına gelebilecek itiraz olarak sorduğu, 00:26:44) Önceden tek bir metot vardı, şimdi dört metot oldu; yönetilmesi gereken parça sayısı artmadı mı? **Cevap:** Evet, bu bir dezavantajdır. Ama elde edilen kazanımların yanında çok küçük kalır; bu yüzden parçalamak tercih edilir.

Soru: (hocanın sorduğu ikinci itiraz, 00:27:19) Metot içinden metot çağırınca her çağrıda bağlam değişimi (context switching) yapılmıyor mu? **Cevap:** Hoca bağlam değişimini şöyle anlattı: çağıran metodun durum bilgisi bir yığına atılır, çağrılan metot için yığının üstüne yeni bir alan eklenir, metottan dönülünce `pop` edilerek önceki metodun değişkenleri ve sayaçları geri gelir ve kaldığı yerden devam edilir. Bunun arka planda yürüttüğü işlere context switching overhead denir. Ama bu kaygı çoğunlukla eskide kalmıştır: modern derleyiciler ve yorumlayıcılar (yorumlanan diller için just-in-time derleyiciler) bu yükü önemli oranda azaltır; hocaya göre ek yük yok denecek kadar azdır.

> **Ders sonrası not:** Hoca çağıran metodun durum bilgisinin "heap'te" bir yığına atıldığını söyledi (00:27:28). Yerel değişkenler ve dönüş adresi çağrı yığınında (stack) tutulur; heap ise nesneler için kullanılan bellek bölgesidir. Hocanın andığı out-of-order execution (birbirine bağımlı olmayan kod bloklarını sırası dışında yürütmek) ise, büyük olasılıkla, derleyicinin değil işlemcinin donanım düzeyinde yaptığı bir iyileştirmedir; metot çağrısının yükünü derleyici ve JIT tarafında asıl azaltan, büyük olasılıkla, metodun çağrıldığı yere gömülmesidir (inlining).

5. Yinelenen Kodu Toplamaya Yönelik Diğer Düzenlemeler

Hoca Duplicated Code'a geri döndü (ekrandan, 00:30:00). Slayttaki "çözüme götürecek diğer düzenlemeler":

- **Pull Up Method:** Yinelenen kod kalıtım ağacındaki kardeş veya kuzen sınıflara yayılmışsa, oluşturulan metot ortak üst sınıfa çekilir.
- **Form Template Method:** Kod aynı değil ancak benzerse, GoF Template Method tasarım kalıbı kullanılarak kardeş/kuzen sınıflarda düzenleme yapılır.
- **Eğer yinelenen veya benzer kod ilişkisiz sınıflarda yer alıyorsa:** sorumluluklar doğru dağıtılmamış olabilir; metot bir sınıfa atanır, diğerleri bu sınıfı kullanır; ya da bu metot oluşturulacak yeni bir sınıfa atanır ve diğerleri bu sınıfı kullanır.
- **Özetle:** Extract Method ile oluşturulan yeni metoda en uygun yer aranır.

Pull Up ya da Push Down gibi adlardaki yukarı ve aşağı, kalıtım hiyerarşisinde üst sınıflara doğru çıkmak ya da alt sınıflara doğru inmek demektir. Bir metot çıkarılıp aynı sınıfta kalırsa bu Extract Method'dur; sınıf bir kalıtım hiyerarşisindeyse ve çıkarılan metot bir ya da daha fazla üst sınıfa yerleştirilirse Pull Up Method'dur.

Kod aynı değil benzer olduğunda küçük varyasyonları modellemek için Template Method kalıbı kullanılabilir. Kalıp daha önce işlendiği için hoca ayrıntıya girmedi: değişmeyen kısımlar üst sınıfta, değişecek kısımlar alt sınıflarda kodlanır; üst sınıftaki şablon metodun içinde alt sınıfta işi yapacak metotların adları verilir. Şablon metodun alt sınıfta ezilememesi gerektiği için `final` tanımlanmalıdır; yoksa kalıp bozulur.

Yinelenen kod birbiriyle ilişkisiz sınıflarda yer alıyorsa bu, sorumlulukların baştan doğru dağıtılmadığının işareti olabilir. Hocaya göre o zaman küçük refactoring adımları yerine tasarımın

o kısmını tamamen değiştiren kapsamlı bir yenileme (complete overhaul) gerekebilir; buna değip değmeyeceğine karar verilir. Bunu yapamıyorsak bile metot kopyaların geçtiği sınıflardan birine konur ve diğerleri onu çağırır; ya da tasarıma yepyeni bir sınıf eklenip metot oraya çıkarılır ve kodun koparıldığı sınıflardan bu yeni yere çağrı yönlendirilir.

Hoca, Fowler'ın anlatımında kod kusurlarının ve refactoring eylemlerinin birbiriyle ilişkili yumaklar gibi olduğunu, bu yüzden konuyu tek bir düzgün akışla anlatmanın zor olduğunu ve zaman zaman geriye gönderme yapılacağını söyledi.

6. Pull Up Method: Metodu Yukarı Çekme

6.1 Tanım ve örnek

Slayttaki tanım (ekrandan, 00:34:40): kardeş sınıflarda yinelenen metotlar varsa, bunlar üst sınıfa çekilerek bir tek yerde kodlanır.

Örnek (UML sınıf şeması, metne dökülmüştür):

```
ÖNCE:                                   SONRA:
        Employee                                Employee
           ^                                    + getName
     +-----+------+                               ^
  Salesman      Engineer                      +-----+------+
  + getName     + getName                   Salesman      Engineer
```

İki ayrı sınıfta aynı biçimde kodlanmış bir metot üst sınıfta tanımlanırsa alt sınıflara kalıtımla zaten geçer; aynı kodu iki kez yazmanın anlamı yoktur.

Hoca şemaların Fowler'ın kitabının elektronik sürümünden ekran görüntüsü olarak alındığını ve Fowler'ın bazı ayrıntıları göstermediğini söyledi. Üye metotlar, çok iç işlerle ilgili olanlar dışında, public tanımlanmalıdır; bir sınıfın iş yapabilmesi için public metotları olmalıdır. Hoca `getName` metodunu dönüş türüyle birlikte `+getName() : String` biçiminde yazılması gerektiğini gösterdi.

> **Hocanın vurgusu:** Bu şemalar kitaptan doğrudan alındığı için ufak tefek UML hataları içerebilir. Sınavda bunları olabildiğince düzgün uygulamaya çalışın (00:36:40).

6.2 Özel durum: çok farklı kodlanmış metotlar

Slayttaki özel durum örneği (ekrandan, 00:40:00), metne dökülmüştür:

- **Önce:** `Customer` sınıfında `addBill(dat: Date, amount: double)` ve `lastBillDate` vardır. Alt sınıflar `RegularCustomer` ve `PreferredCustomer`'ın her ikisinde `createBill(Date)` ve `chargeFor(start: Date, end: Date)` metotları vardır. Şemadaki notlar `chargeFor` metotları için "Çok farklı kodlanmışlar" der.
- **Sonra:** `Customer` sınıfında `lastBillDate` üye alanı ile `addBill(dat: Date, amount: double)`, `createBill(Date)` ve italik (soyut) `chargeFor(start: Date, end: Date)` metotları vardır. Alt sınıflarda yalnız `chargeFor(start: Date, end: Date)` kalır. Not: "chargeFor metodu üst sınıfta soyut tanımlandı".
- Slaytın altındaki madde: özel durumlarda Template Method tasarım kalıbı kullanılabilir.

Hoca şema üzerinden UML gösterimine dair birkaç uyarı yaptı:

- `Customer` sınıfının adı italik yazılmıştır, yani soyut bir sınıftır. İtalik yazmak yerine adın üstüne `<<abstract>>` yazmak çok daha iyi olurdu. Aynı şekilde soyut metot da yalnız italikle gösterilmiştir; kâğıt kalemle yazarken italik anlaşılmaz, metodun sonuna "abstract" yazarak göstermek daha yararlıdır.
- Önceki haldeki `Customer` kutusunda metotlar ortada, üye alanlar altta gösterilmiştir. Java kodunda sıra sorun olmaz, ama hoca UML'de önce üye alanların, sonra metotların gösterilmesini istiyor ve bunu alışkanlık haline getirmeyi öneriyor.
- Üye alanların private, metotların public olduğu gösterilmemiştir; bunlar da gösterilmelidir. Hoca slaytlarda yorum kutularını Türkçeleştirdiğini ama diğer ayrıntıları düzeltmeye üşendiğini, öğrencilerin bunları yapmaması gerektiğini söyledi.

Senaryo: normal bir müşterinin ve tercih edilen (premium) bir müşterinin ücretlendirilmesi farklıdır; aralarında istisnalar, kampanyalar gibi ayrıntılar vardır. Bu yüzden `chargeFor` iki alt sınıfta çok farklı kodlanmıştır ve içi ortak kodlanamaz. Çare, `chargeFor`'u üst sınıfta soyut metot olarak tanımlamaktır. Böylece hiç olmazsa metot imzası yukarıda tanımlanmış olur; bu, alt sınıflarda gövdenin mutlaka tanımlanması gerektiğine dair bir uyarıdır.

Bu sayede ister `RegularCustomer` ister `PreferredCustomer` olsun, her müşteri `Customer` gibi erişilip faturalandırılabilir. Ortak kod pek olmadığı için gövdeler ayrı sınıflarda kalır, ama hepsine `Customer` olarak erişilebilmesi yeterli bir kazançtır. Bu da yetmiyorsa Template Method kullanılır; hoca bunu kalıplar konusunda yeterince konuşulduğu için tekrar göstermedi.

7. Aşırı Uzun Kodu Kısaltmaya Yönelik Diğer Düzenlemeler

Uzun metodu kısaltmanın Extract Method dışında üç yolu daha vardır (ekrandan, 00:43:30):

- **Decompose Conditional:** Karar verme (if-switch-case) ve döngülerin yoğun olması çoğu kez kodu uzatır.
- **Replace Temp with Query:** Geçici yerel değişkenlerin kullanımı genelde aynı geçici değişkenin kullanımı bahanesi ile kodun uzamasını teşvik eder.
- **Replace Method with Method Object:** Üstteki düzenlemeler geçici değişkenleri ve uzun parametre listesini yok edememişse kullanılabilecek daha karmaşık bir düzenleme.

Decompose Conditional'daki "condition" if, else-if, else ya da switch-case yapılarıdır; "decompose" ise birleştirmenin (kompozisyonun) tersi, parçalarına ayırmadır. Bu eylem, karar yapılarını kimi zaman tamamen yok etmeye, kimi zaman etkilerini önemli ölçüde azaltmaya kadar gider.

Replace Temp with Query, geçici değişkenler yerine metot çağrısı yapılmasını önerir. Programcı bir hesabın sonucunu saklayıp tekrar tekrar kullanmak için geçici değişken tanımlar; veri bayatlamamışsa son saklanan sonuç kullanılabilir. Ama ara sonucu orada da burada da kullanayım derken ve bir şey çıkarılmadığında metot uzar gider. Toparlarsak, geçici yerel değişkenlerin ya da nesnelerin kullanımı kodu uzatmaya meylettirir.

7.1 Zaman-bellek ödünleşimi

Hoca geçici değişkenin bir zaman-bellek ödünleşimi (time-memory trade-off) olduğunu anlattı (00:44:48). İşlem sonucunu geçici bir değişkende saklamak, parametreler değişmeden hesap yeniden gerektiğinde onu tekrar yapmayı önler; kodun çalışma süresini düşürür, yani kodu hızlandırır. Öte yandan her geçici değişken bellekte ayrı yer kapladığı için bellek ihtiyacını artırır.

Hangisi projeniz için daha sakıncalıysa ona göre seçim yapılır. Ama hocaya göre başarım darboğazları genelde buralardan değil, daha ciddi yerlerden kaynaklanır. Bu yüzden mümkünse geçici değişken kullanılmamalıdır. Hiç kullanmamak da mümkün değildir: örneğin birikimli toplam yapılacaksa, adı üstünde, birikimin tutulacağı bir geçici değişken gerekir.

8. Decompose Conditional: Karar Verme Komutunu Parçalama

8.1 Önceki hal ve koşulun çözümlenmesi

Slayttaki örnek (ekrandan, 00:50:00):

```
/*** ÖNCE: ***/
if (date.before (SUMMER_START) || date.after(SUMMER_END))
    charge = quantity * _winterRate + _winterServiceCharge;
else charge = quantity * _summerRate;
```

Slayttaki amaç: aşırı karmaşık if komutlarını sadeleştirmek anlaşılabilirliği artırır. Hocaya göre anlaşılması zor kodda hata bırakma olasılığı da yükselir.

Kod şöyle okunur: `date` bir yerel değişkendir (alt çizgiyle başlamıyor). `SUMMER_START` ve `SUMMER_END` büyük olasılıkla tarih türünden sabitlerdir. Tarih yaz başlangıcından önceyse veya yaz sonundan sonraysa ücret, miktar çarpı kış oranı (`_winterRate`, alt çizgili, yani üye alan) artı

bir kış servis ücretidir; kış koşullarında yapılan tamir ve bakım işleri daha masraflıdır. Aksi halde ek ücret olmadan miktar çarpı yaz oranı uygulanır.

Kış ücretinin uygulanması koşulun kışı anlattığını düşündürür, ama hocaya göre "programcı biraz paranoyak olmalı" ve her şeyi kontrol etmelidir. Hoca koşulu bir zaman çizgisi üzerinde çizerek çözümledi:

```
------------|=========== YAZ ===========|------------>  zaman
       SUMMER_START                 SUMMER_END
 before(SUMMER_START)                    after(SUMMER_END)
 --> kış ücreti                          --> kış ücreti
```

Yaz başlangıcından önceki ve yaz sonundan sonraki tarihler yaz olmayan dönemleri tarar; ortadaki kısım yazdır. Yani "kış servis ücreti" demek mantıklıdır. İlkbahar ve sonbaharda da kış ücreti alınıyor; bu bir hata mı diye sorulabilir, ama iş mantığı ve satış kararları böyleyse kod doğru kabul edilir. Yine de koşulu anlamakta bile biraz zorlanıldı; bu karmaşık koşul parçalara ayrılabilir mi?

8.2 Sonraki hal

```
/*** SONRA: ***/
if (notSummer(date))
    charge = winterCharge(quantity);
else charge = summerCharge (quantity);
private boolean notSummer(Date date) {
    return date.before (SUMMER_START) ||
    date.after(SUMMER_END);
}
private double summerCharge(int quantity) {
    return quantity * _summerRate;
}
private double winterCharge(int quantity) {
    return quantity * _winterRate + _winterServiceCharge;
}
```

Kod parçadır, derlenmedi.

Koşul `notSummer` adlı bir metoda çıkarılmıştır. Bu metodu dışarıdan başka birinin çağırmasının anlamı olmadığı için `private` tanımlanmıştır. Koşul doğruysa `true`, değilse `false` döner. Hoca bunun karmaşıklığı "halının altına süpürmek" gibi göründüğünü kabul etti: koşul tam parçalanmadı, ama artık işin içinde değil, ayrı ve odaklanmış bir metodun içindedir. Üst kısma dönüldüğünde "yaz değilse ücret kış ücreti, değilse yaz ücreti" diye ne yapıldığı hemen anlaşılır. Önceki halde ne yapıldığına dair en ufak fikir için bile koşulu ayrıntılı okumak gerekiyordu.

İki daldaki hesaplamalar da ayrı alt işler sayılır ve `summerCharge` ile `winterCharge` metotlarına çıkarılır. Hesaplama yöntemlerinde bir değişiklik gerekmez. Böylece hem Extract Method hem Decompose Conditional yapılmış olur.

Hoca kendi tercihini de söyledi (00:55:30): yaz olmayan kısımları taramak yerine "tarih yaz başlangıcından sonra ve yaz sonundan önceyse `false`, değilse `true` döndür" biçiminde yazmak ona daha öz ve anlaşılır gelir. Öbür biçimi daha rahat anlayan için o da kalabilir.

> **Ders sonrası not:** Hocanın önerdiği `after(SUMMER_START) && before(SUMMER_END)` biçimi sınır günlerde slayttaki koşulla aynı sonucu vermez. Tarih tam `SUMMER_START` ya da tam `SUMMER_END` ise slayttaki `notSummer` `false` (yaz) döndürür, önerilen biçim `true` (yaz değil) döndürür. Davranışı korumak refactoring'in temel şartı olduğundan eşdeğer yazım `return !(!date.before(SUMMER_START) && !date.after(SUMMER_END));` olmalıdır. Bu fark, tarihler tamsayıyla temsil edilerek python ile denetlendi.

9. Long Parameter List: Uzun Parametre Listesi

Aradan sonra üçüncü kod kusuruna geçildi (ekrandan, 01:12:00). Slayttaki tanım:

- Bir metodun parametre sayısının fazla olması, hatta bu parametrelerden bazılarının başka metotlarda da birlikte geçmesi durumudur.
- Demek ki aslında bu parametreler birlikte bir varlığın durum bilgisinin bir kısmını (1) veya tamamını (2) oluşturmaktadır.
- Çözüme götürecek düzenleme: (1) Introduce Parameter Object, metot parametrelerinin sayısı çok ise azaltmaya yönelik bir düzenlemedir; (2) Preserve Whole Object, metot parametrelerinin sayısı çok ise azaltmaya yönelik diğer bir düzenlemedir.

Hoca bir iki parametreyi olağan, daha fazlasını şüphe uyandırıcı buluyor. Diğer metotların parametrelerine bakınca listelerin bir yere kadar ortak olduğu da görülebilir. Bu, aslında bir varlığın durum bilgisinin bir kısmının ya da tamamının metot parametresi olarak, onu ilgilendiren başka sınıflara ayrı ayrı gönderildiği anlamına gelir. Birinci durumda Introduce Parameter Object, ikinci durumda Preserve Whole Object kullanılır.

10. Introduce Parameter Object: Parametre Nesnesi Oluşturma

Slayttaki tanım (ekrandan, 01:20:00): bir metot çok fazla parametre alıyorsa, uygun parametreleri üye alan olarak yeni bir sınıfta toplayın ve o türden bir nesneyi parametre olarak kullanın.

Örnek (UML sınıf şeması, metne dökülmüştür):

```
ÖNCE: Customer                                    SONRA: Customer
  amountInvoicedIn(start: Date, end: Date)          amountInvoicedIn(DateRange)
  amountReceivedIn(start: Date, end: Date)  =>      amountReceivedIn(DateRange)
  amountOverdueIn(start: Date, end: Date)           amountOverdueIn(DateRange)
```

Buradaki parametre listesi aslında uzun değildir, ama birçok metotta tekrarlanır. Bu, iki alanın çoğunlukla birlikte kullanıldığını ve bir araya gelip başlı başına bir varlık oluşturabileceğini gösterir: başlangıç ve bitiş aslında bir zaman aralığıdır. Bu iki parametre üye alan olacak biçimde `DateRange` adlı bir sınıf oluşturulur ve metotlar onu parametre olarak alır.

Slayttaki ek maddeler: bunu sadece parametre listesini kısaltmak amacıyla yapmış olmazsınız. Oluşturulan `DateRange` sınıfı sadece iki veriyi birleştiren bir kayıt yapısı şeklinde kalmaz; `isWithinRange(Date)` gibi uygun metotları da içererek "decompose conditional" veya "extract method" düzenlemesi de yapılmış olunur.

Hoca bunu önceki ders saatinin sonundaki örnekle bağladı: bir tarihin belli bir aralığın içinde mi dışında mı olduğuna dair hesaplar, bunların geçtiği her yerde ayrı ayrı kodlanıp tekrarlanmak yerine yeni sınıfta toplanır. Hiçbir şey olmasa bile daha derli toplu kod yazılmış olur. O varlıkla ilgili alt işler için de, yani çıkarılan metotları koymak için de, iyi bir yer ortaya çıkar.

11. Preserve Whole Object: Tüm Nesneyi Gönder

11.1 Tanım ve örnek

Slayttaki tanım ve örnek (ekrandan, 01:16:40):

```
/*** ÖNCE: ***/
int low = daysTempRange().getLow();
int high = daysTempRange().getHigh();
withinPlan = plan.withinRange(low, high);

/*** SONRA: ***/
withinPlan = plan.withinRange(daysTempRange());
//withinRange metodu da gerektiği gibi değiştirilir.
```

Kod parçadır, derlenmedi.

Bir nesnenin çeşitli üyelerini aynı metodun farklı parametreleri olarak göndermek yerine nesnenin kendisi gönderilir. Önceki halde `daysTempRange()` metodu bir nesne döndürür; onun `getLow()` metoduyla `low` geçici değişkeni, `getHigh()` metoduyla `high` geçici değişkeni elde edilir ve bu iki `int` `plan.withinRange` metoduna verilir. Sonraki halde `withinRange` metodu,

`daysTempRange()` hangi türden nesne döndürüyorsa o türden bir parametre alacak biçimde değiştirilir; düşük ve yüksek değerlerin sorgulanması artık onun içinde yapılır.

Hoca bir yarar daha ekledi: belki `withinRange` metodunun o nesnenin başka üyeleriyle de ilgilenmesi gerekecektir ama bu henüz fark edilmemiştir. Burada yalnız iki alan gönderiliyor; ileride üç, dört, beş ayrı veri gerekebilir. Tüm nesne gönderildiğinde parametre listesinin yeniden uzamasının önü kesilmiş olur.

Hoca sonraki hale bakarak bu eylemin "bir taşla iki kuş vurduğunu" söyledi (01:21:15): önceki halde `low` ve `high` geçici değişkenleri vardı, sonraki halde yoktur. Yani eylem hem uzun parametre listesini toparlar hem de pek sevilmeyen geçici değişkenleri ortadan kaldırır. Geçici değişkenleri kaldırmaya yönelik asıl eylem, Long Method için birazdan görülecek olan Replace Temp with Query'dir.

> **Ek bilgi (derste anlatılmadı):** Hoca `daysTempRange()`'in "herhalde DateRange türünden" bir nesne döndürdüğünü tahmin etti (01:19:20). Fowler'ın kitabındaki bu örnekte, büyük olasılıkla, bir odanın günlük sıcaklık aralığı söz konusudur ve metot `TempRange` türünden bir nesne döndürür; `withinRange` de bu türden bir parametre alacak biçimde değiştirilir. Tür adı ne olursa olsun eylemin mantığı aynıdır.

11.2 Zayıf yönler

Slayttaki maddeler (ekrandan, 01:16:40):

- Bu düzenlemenin 1. zayıf yönü: bağımlılık artabilir. Parametre olarak gönderilen nesnenin sınıfı ile metodun sahibi olan sınıf arasında bir bağımlılık oluşur.
- Bu düzenlemenin 2. zayıf yönü: geriye doğru uyumluluk kaybedilerek refactoring tanımı esnetilmiştir, çünkü söz konusu metotların imzası değişmiştir.
- Bu durumlardan biri bile çok sakıncalı olacaksa bu düzenlemeyi kullanmayın.

Hoca birinci zayıf yönü şema üzerinden anlattı. Yeni bir `DateRange` sınıfı oluşturulduğunda, sahiplik olmasa bile en azından bir kullanma ilişkisi kurulur. Daha önce olmayan bir sınıf ortaya çıkarılıp `Customer` ile arasında ilişki kurulunca bağlaşım artar. Kullanma ilişkisi sahiplik kadar ağır değildir; hoca onu "yarım ağırlıkla" düşünmeyi önerdi. Ama az ya da çok, eskiden olmayan bir ilişki tasarıma eklenmiş ve bağlaşım yükselmiştir. Düşük bağlaşım özellikle hedefleniyorsa, parametre listesi uzun kalsın denip eylem uygulanmayabilir.

İkinci zayıf yön refactoring'in tanımıyla ilgilidir. En genel tanım "yazılımın ne yaptığını değiştirmeden nasıl yaptığını değiştirmek"tir; bu açıdan bakınca sorun görülmeyebilir. Ama refactoring "yazılımın dış dünyaya açılan penceresini, yani arayüzlerini değiştirmeden iç yapısını ve tasarımını değiştirmek" olarak da algılanabilir. Bu tanım geriye doğru uyumluluğun korunduğu tanımdır ve bazı durumlarda geriye doğru uyumluluk çok önemlidir. Bu eylem ise public bir metodun imzasını değiştirir, dolayısıyla geriye doğru uyumluluk kaybedilir. Bu iki olumsuzluktan biri ya da ikisi önemliyse eylem yapılmaz; önemli değilse yapılır.

> **Ders sonrası not:** Hoca bu zayıf yönleri anlatırken eylemin adını "Introduce Parameter Object" diye andı (01:20:48, 01:23:51) ve örnekte `DateRange` sınıfını kullandı. Ekrandaki slaytta bu iki zayıf yön Preserve Whole Object başlığı altında yazılıdır (ekrandan, 01:16:40). Her iki eylem de public metot imzasını değiştirdiği ve yeni bir sınıfa bağımlılık getirdiği için, büyük olasılıkla, söylenenler ikisi için de geçerlidir.

12. Replace Temp with Query: Geçici Değişken Yerine Erişim Metodu

12.1 Tanım ve getPrice örneği

Slayttaki tanım (ekrandan, 01:30:00): bir ifadeyi bir geçici değişkende saklayarak kullanmak yerine, ifadeden metot(lar) çıkartın.

```
/*** ÖNCE: ***/
double getPrice() {
    int basePrice = _quantity * _itemPrice;
    double discountFactor;
    if (basePrice > 1000) discountFactor = 0.95;
    else discountFactor = 0.98;
```

```
    return basePrice * discountFactor;
}

/*** SONRA: ***/
double getPrice() { return basePrice() * discountFactor(); }
private int basePrice() { return _quantity * _itemPrice; }
private double discountFactor() {
    if (basePrice() > 1000) return 0.95;
    else return 0.98;
}
```

Kod parçadır, derlenmedi.

Önceki halde `getPrice` bedel hesaplama metodudur. `basePrice` geçici değişkeni miktar çarpı birim fiyattır (`_quantity` ve `_itemPrice` alt çizgili, yani üye alan). `discountFactor` geçici değişkeni, `basePrice` 1000'den büyükse 0.95, değilse 0.98 olur. Sonunda temel fiyata indirim katsayısı uygulanıp döndürülür.

Hoca geçici değişkenlerin geçtiği yerlerin altını çizerek inceledi (IDE'deki vurgulama özelliği burada olmadığı için). `basePrice` bir kez hesaplandıktan sonra hiçbir yerde değiştirilmeden kullanılmıştır. `discountFactor` da iki daldan birinde bir kez değer aldıktan sonra kullanılırken başka bir değişikliğe uğramaz.

> **Hocanın vurgusu:** Geçici değişkene yalnız bir kez değer atanıp sonra bir ya da daha fazla yerde değiştirilmeden kullanılıyorsa onu ortadan kaldırmak çok daha kolaydır. Arada bir yerlerde değişiyorsa, kaç yerde değiştiğine göre o kadar ayrı Extract Method yapılabilir; tek bir metot çıkarmak zorunlu değildir (01:27:09).

Değeri değişen geçici değişkeni ortadan kaldırmak daha fazla çaba ister. Hoca zaman varsa bu çabanın harcanmasını tercih ediyor; refactoring için ayrılan zaman kısıtlıysa bu adım atlanabilir.

Sonraki halde `getPrice` "temel fiyat çarpı indirim katsayısı" diye tek satırda okunur. `basePrice()` miktar çarpı birim fiyatı döndürür. `discountFactor()` temel fiyata bağlı olduğu için onun içinde `basePrice()` yeniden çağrılır.

12.2 Çarpma sayısı ve başarım

Hoca akışı izleyerek işlem sayısını karşılaştırdı (01:29:12):

- **Önceki hal:** `_quantity * _itemPrice` (1 çarpma) ve `basePrice * discountFactor` (1 çarpma); toplam 2 çarpma.
- **Sonraki hal:** `getPrice` içinde `basePrice()` çağrılır (1 çarpma), `discountFactor()` içinde `basePrice()` yeniden çağrılır (1 çarpma daha), sonuçlar çarpılır (1 çarpma); toplam 3 çarpma.

Hoca çarpmanın toplama ve çıkarmaya göre daha masraflı bir işlem olduğunu, ama saniyede milyonlarca işlem yapan bilgisayarlarda bir iki fazla çarpmanın hissedilmeyeceğini söyledi.

Ancak geçici değişkenler işlem tekrarını önler. Karmaşık bilgi sistemlerinde bazı hesaplar farklı sunucu adacıklarında, belki ağ üzerinden dış ağa çıkılarak buluttan çağrılarak yapılır. Böyle geciktirici bir hesabın gereksiz yere birden fazla yapılması başarımı önemli ölçüde düşürebilir. Hoca, bazı öğrencilerin bu noktayı itiraz olarak getirdiğini aktardı: sonucu başka bir makinede uzun hesaplama süresi, enerji maliyeti ve haberleşme masrafıyla elde ediyorlarsa onu saklamaları gerekir. Hocanın cevabı: öyle bir durumda bu eylemi yapmayın; öyle bir durum yoksa birkaç fazla işlem karşılığında çok daha kolay anlaşılan ve bakımı kolay bir kod elde edersiniz.

12.3 Avantaj

Slayttaki maddeler (ekrandan, 01:32:40):

- Avantaj: kopyala-yapıştır kodlamayı daha ortaya çıkmadan azaltmak. Geçici değişkeni kullanan metodunkine benzer bir işlem eklenmek istendiğinde mevcut metottan yeni metoda kopyala-yapıştır yapılacak. Bu kod parçacığı ile ilgili bir hata bulunursa tüm sınıfta

buradan kopyala-yapıştır yapılan yerler aranıp her yerde düzeltme yapmak gerekecek. Halbuki geçici değişkenlerle yapılan işlemler sorgu metotlarında toplanırsa, düzeltmeler de sadece sorgu metotlarında yapılacaktır.

- Tartışma: bu düzenleme başarımı düşürür; sadece işlem yapmak yerine bir de fazladan metot çağırma yükü geliyor.
- Şerh: modern derleyiciler bu ek yükü ortadan kaldırmış veya önemsizleştirmiştir.

Hoca avantajı şöyle açtı: işin başında hesap bir sorgu metoduna dönüştürülürse, gereken bütün yerlerde yalnız o metot çağrılır ve yinelenen kod daha ortaya çıkmadan engellenir. Bir hata ya da yeni gereksinim çıkarsa değişiklik o odaklanmış sorgu metodunda yapılır; büyük olasılıkla uzun bir metodun içinde ilgili bölgeyi aramak gerekmez.

12.4 Tartışma: geçici değişken birden fazla yerde kullanılıyorsa

Slayttaki tartışma maddesi (ekrandan, 01:40:00): geçici değişken metot içerisinde birden fazla yerde kullanılıyorsa aynı işlem tekrar tekrar yapılacak; bu da başarımı daha da fazla düşürecek.

```
void printRecipt() { /*** ÖNCE: ***/
   int totalAmount = 0, thisAmount;
   for( Item item : items ) {
      thisAmount = item.getCharge( );
      System.out.println(item.getName()+" "+thisAmount);
      totalAmount += thisAmount;
   } }

void printRecipt() { /*** SONRA: ***/
   int totalAmount = 0;
   for( Item item : items ) {
      System.out.println(item.getName()+" "+item.getCharge());
      totalAmount += item.getCharge( );
   } }
```

Kod parçadır, derlenmedi. Metot adı slayttaki gibi (`printRecipt`) yazılmıştır.

Önceki halde iki geçici değişken vardır: toplam tutar (`totalAmount`) ve tek bir kalemin tutarı (`thisAmount`). Döngüde önce kalemin ücreti alınıp `thisAmount`'a atanır, ekrana yazılır, sonra birikimli toplama eklenir. `totalAmount`'a dokunulamaz, çünkü birikimli toplam için geçici değişken şarttır. `thisAmount` ise bir kez `item.getCharge()` ile değer alır ve sonra değişmeden iki yerde kullanılır; ortadan kaldırmak zor değildir. Hoca yöntemi şöyle anlattı: değişken silinince kullanıldığı iki yerde derleme hatası alınır; `thisAmount` doğrudan `item.getCharge()` değeri olduğu için bu iki yere `item.getCharge()` yazılır ve sonraki hale ulaşılır.

Artık `getCharge` iki kat çağrılır. Hocanın hesabı (01:37:31):

- `getCharge` 1 ms sürsün. 100 kayıtta önceki hal $100 \times 1 = 100$ ms harcar.
- Sonraki hal her kayıt için iki çağrı yapar: $100 \times 2 = 200$ ms.
- Fark 100 ms'dir; bu hissedilmez. Bu yüzden eylem yapılabilir.

Hesaplama çok masraflıysa, başka sunucularda hesaplanıp geri dönen hesaplar gibi, eylem yapılmaz.

12.5 Profiler, geri alma ve darboğaz

Bunun önceden fark edilemeyebileceğini söyleyen hoca profiler araçlarını anlattı (01:38:35). Profiler kimi zaman kod kapsamını (code coverage) hesaplamak için, kimi zaman başarım sınaması için kullanılır: bir metot çağrıldığında tamamlanması ne kadar sürdü, içinden hangi metotlar çağrıldı, onların her biri tek tek ve toplamda ne kadar zaman harcadı, bunları raporlar.

Önerilen yol:

1. Refactoring yapmadan önce başarımı ölçün.
2. Refactoring'den sonra başarımı yeniden ölçün.

3. Fark hissedilir düzeydeyse, büyük bir orandaysa ya da önceden belirlenen sınırları aşıyorsa, yapılan değişiklikleri adım adım geri izleyerek refactoring eylemlerini geri alın.
4. Daha iyisi, bir yazılım yapılandırma yönetimi (software configuration management) aracı kullanılıyorsa, eylemlerden önceki ve sonraki sürümler ayrı sürümler olarak durur; başarım kabul edilenin altına düşerse önceki sürüme dönülür (rollback).

Hoca "zincir en zayıf halkası kadar güçlüdür" deyimini hatırlattı; kişiler ya da sistem parçaları için kısaca "zincirin zayıf halkası" denir. Bilgi sistemlerinde de belirli bir noktada darboğaz (bottleneck) olabilir. Hoca bunu trafik şeritleriyle anlattı: Boğaz Köprüsü'nden önce yol beş şerittir, köprüde üç şeride iner; beş şeritten üçe geçerken ister istemez bekleme olur ve araç yoğunluğuna göre az ya da çok sürer. Darboğaz kaldırılırsa akış çok daha iyi olur. Aynısı bilgi sistemlerindeki denetim, veri ve işlem akışı için de geçerlidir. Bir refactoring eylemi darboğazı kötüleştirdi mi diye bakmak gerekir.

Ama büyük olasılıkla darboğaz refactoring yapılan yerlerin dışındadır; tasarımı ve kodu yapanlar oraların neresi olduğunu zaten bilir. Hocanın örneği (01:41:53):

- Bir eylemden 100 ms, başka bir eylemden 200 ms, iki eylemden de 50'şer ms ek yük gelsin: toplam $100 + 200 + 50 + 50 = 400$ ms.
- Sistemde işlemleri 3 saniye, yani 3000 ms zorlayan başka bir darboğaz olsun.
- 3000 ms'nin yanında bu ek yük göz ardı edilebilir. Buna karşılık bakım kolaylığı ve bunun getireceği maddi tasarruf büyüktür; bu yüzden eylem yapılır.

  Ders sonrası not: Hoca toplam ek yükü 400 ms diye hesapladıktan sonra karşılaştırmada "3000'in yanında 200" dedi (01:42:13). Hesaba göre karşılaştırılması gereken ek yük 400 ms'dir; sonuç değişmez, 400 ms de 3000 ms'nin yanında küçüktür.

Hoca darboğazın orada kalmasının büyük olasılıkla bir nedeni olduğunu, kaldırılabilseydi kaldırılacağını ya da geçici bir çözüm yapılıp sonraki sürümlerde ele alınabileceğini söyledi. Mevcut sürümde, darboğazın yanına bile yaklaşmayan küçük ek yükler getiren refactoring eylemlerini toplamda yapmanın sakıncası yoktur.

13. Replace Method with Method Object: Metot Yerine Yeni Türden Nesne

Aynı kusura, yani Long Method'a yönelik son eylem (ekrandan, 01:43:50). Slayttaki maddeler:

- Metot öyle uzun ve geçici yerel değişkenleri öyle arap saçı gibi kullanıyor ki "extract method" düzenlemesini kullanamıyorsunuz.
- O zaman yeni bir sınıf oluşturup bütün metodu o sınıfa taşıyın, yerel değişkenleri de o sınıfın üyeleri yapın.
- Ardından bildiğiniz düzenleme eylemlerini daha rahat yürütebilirsiniz.
- Yeni sınıfın adını verirken, taşınan metodun adına benzer bir ad verin.

Önceki hal:

```
/*** ÖNCE: ***/
class Order {
   double price() {
   double primaryBasePrice;
   double secondaryBasePrice;
   double tertiaryBasePrice;
   // long computation;
   ...
}
```

Kod parçadır, derlenmedi; slaytta da kapanış ayraçları eksiktir.

Sonraki hal (UML sınıf şeması, metne dökülmüştür):

```
+-----------+                       +--------------------+
|   Order   | - - - - - - - - - - > |  PriceCalculator   |
+-----------+                       +--------------------+
|           |                       | primaryBasePrice   |
+-----------+        1              | secondaryBasePrice |
```

```
| price()   | <------------------- | tertiaryBasePrice  |
+-----------+                      +--------------------+
     :                             | compute()          |
     :                             +--------------------+
  not: return new PriceCalculator(this).compute()
```

`Order`'dan `PriceCalculator`'a kesikli bir bağımlılık oku, `PriceCalculator`'dan `Order`'a 1 çokluklu bir ilişki vardır; `price()` metoduna bağlı not `return new PriceCalculator(this).compute()` der.

Hoca senaryoyu şöyle canlandırdı: `Order` sınıfının `price` metodunda en az üç geçici değişken ve uzun hesaplamalar vardır. Değişkenler oluşturulur, birkaç kez kullanıldıktan sonra değeri değiştirilir, hesap tazelenir ya da üstüne küçük bir ek yapılır, o ek başka yerlerde yeniden kullanılır, diğer ikisi toplanıp çarpılır. Böyle bir metot refactor etmek için "çetin ceviz"dir. O zaman bu zor kısım için `PriceCalculator` adlı yeni bir sınıf oluşturulur, geçici değişkenler onun üye alanları olur ve örneğin `compute` adlı bir metot eklenir. Hoca sınıf adı `PriceCalculator` olduğu için metoda `calculate` demenin daha iyi bir ad olabileceğini sonradan fark ettiğini söyledi.

`Order` sınıfının `price` metodunda artık yapılan tek şey, bir `PriceCalculator` nesnesi oluşturup ona `Order` nesnesinin kendisini vermektir. Çünkü büyük olasılıkla `price` metodunun içinde `Order`'ın üye alanlarına erişim (erişim metotlarıyla ya da doğrudan adlarıyla) vardı. Private üye alanların yeni sınıfa aktarılması gerekir, ama hoca bunu tek tek yapmamayı istedi; yoksa uzun parametre listesi ortaya çıkar. Bunun yerine `Order` kendini `this` diyerek bütün olarak, Preserve Whole Object yaparak, `PriceCalculator`'ın yapıcısına ya da uygun görülürse `compute` metoduna parametre olarak verir. Metoda çok müdahale edilemese bile `Order` sınıfının anlaşılabilirliği iyileşmiş olur.

Dezavantaj: `PriceCalculator`, kendisine verilen `Order` nesnesini tutmak için `Order` türünden bir üye alana sahip olur. Sınıf sayısı bir artar; bağlaşım da, hocanın deyişiyle, öbür taraftaki ok da sayılırsa "bir buçuk" artar. Belki daha sonra `PriceCalculator` öyle iyi refactor edilir ki yeniden `Order`'ın içine alınır ve bağlaşım artışı geri alınır.

> **Hocanın vurgusu:** Bazı refactoring eylemlerinin kazanımları net olsa da dezavantajları da vardır. Kazanım ile kayıp birlikte görülüp hangi durumda daha kârlı olunacağına göre yol seçilmelidir (01:48:24).

Örnek Sorular

Soru 1 (derste sözlü, 00:17:51): `printOwing` örneğinde önceki halde yalnız üç komut vardı, Extract Method sonrasında iki metot oldu ve kod toplamda uzadı. Bu durumda Extract Method bir kazanç mıdır?

Cevap: Hocanın cevabı: bu kadar kısa bir örnekte itiraz haklıdır; örnek yalnız eylemin nasıl yapıldığını göstermek içindir. Kazanç uzun metotlarda belirginleşir. Yine de bu örnekte bile yorum satırı gereksizleşmiş, metot adlarından ne yapıldığı anlaşılır hale gelmiş ve bir sorun çıktığında bakılacak yer ikiye bölünmüştür.

Soru 2 (derste sözlü, 00:26:44 ve 00:27:19): Extract Method ile tek metot dört metoda bölününce yönetilecek parça sayısı ve her çağrıdaki bağlam değişimi yükü artar. Bu, eylemden vazgeçmek için yeterli bir neden midir?

Cevap: Hocanın cevabı: hayır. Parça sayısının artması bir dezavantajdır ama anlaşılabilirlik, bakım kolaylığı ve yeniden kullanım kazanımlarının yanında küçük kalır. Bağlam değişimi yükü (context switching overhead) ise çoğunlukla eskide kalmıştır; modern derleyiciler ve JIT derleyicili yorumlayıcılar bu yükü yok denecek kadar azaltmıştır.

Soru 3 (ekrandan, 00:50:00; derste sözlü, 00:48:35): `if (date.before(SUMMER_START) || date.after(SUMMER_END))` koşulu hangi tarihleri kapsar? Decompose Conditional ile nasıl sadeleştirilir?

Cevap: Hoca koşulu zaman çizgisi üzerinde çözümledi: yaz başlangıcından önceki ve yaz sonundan sonraki tarihler, yani yaz olmayan bütün dönemler (ilkbahar ve sonbahar dahil). Bu tarihlerde kış ücreti (`quantity * _winterRate + _winterServiceCharge`), diğerlerinde yaz

ücreti (`quantity * _summerRate`) uygulanır. Sadeleştirmede koşul `private boolean notSummer(Date date)` metoduna, iki daldaki hesaplar `winterCharge(int quantity)` ve `summerCharge(int quantity)` metotlarına çıkarılır; ana kod `if (notSummer(date)) charge = winterCharge(quantity); else charge = summerCharge(quantity);` olur.

Soru 4 (ekrandan, 01:30:00; derste sözlü, 01:29:12): `getPrice()` metoduna Replace Temp with Query uygulanınca bir çağrıda yapılan çarpma sayısı nasıl değişir?

Cevap: Hocanın cevabı: önceki halde 2 çarpma (`_quantity * _itemPrice` ve `basePrice * discountFactor`), sonraki halde 3 çarpma yapılır, çünkü `basePrice()` hem `getPrice()` hem `discountFactor()` içinde çağrılır. Fark modern bir bilgisayarda hissedilmez; ancak hesap çok masraflıysa (örneğin başka bir sunucuda ağ üzerinden yapılıyorsa) eylem yapılmamalıdır.

Soru 5 (ekrandan, 01:40:00; derste sözlü, 01:37:10): `printRecipt()` örneğinde `thisAmount` kaldırılınca `getCharge` her kalem için iki kez çağrılır. `getCharge` 1 ms sürerse 100 kayıtta ek yük ne olur? Bu önemli bir başarım sorunu mudur?

Cevap: Hocanın cevabı: önceki halde $100 \times 1 = 100$ ms, sonraki halde $100 \times 2 = 200$ ms harcanır; ek yük 100 ms'dir ve hissedilmez. Kuşku varsa refactoring'den önce ve sonra profiler ile ölçülür; fark kabul edilen sınırı aşarsa eylem geri alınır ya da sürüm denetim sisteminde önceki sürüme dönülür.

Soru 6 (ekrandan, 01:16:40): Preserve Whole Object düzenlemesinin iki zayıf yönü nedir? Hangi durumda bu düzenleme kullanılmaz?

Cevap: Slayttaki ve derste anlatılan cevap: (1) bağımlılık artabilir; parametre olarak gönderilen nesnenin sınıfı ile metodun sahibi sınıf arasında, sahiplik olmasa da en azından bir kullanma ilişkisi oluşur ve bağlaşım yükselir. (2) Metodun imzası değiştiği için geriye doğru uyumluluk kaybedilir; refactoring "dış dünyaya açılan arayüzü değiştirmeden iç yapıyı değiştirmek" olarak tanımlanırsa bu tanım esnetilmiş olur. Bu durumlardan biri bile çok sakıncalıysa düzenleme kullanılmaz.

Dersin Toparlaması

Bu derste Fowler'ın kataloğundan üç kod kusuru ve bunları gideren refactoring eylemleri işlendi. Duplicated Code, aynı ya da çok benzer kodun birden fazla yerde tekrarlanmasıdır; Extract Method ile kod bir metotta toplanır, kod kardeş sınıflara yayılmışsa Pull Up Method ile üst sınıfa çekilir (çok farklı kodlanmışsa üst sınıfta soyut tanımlanır), benzerse Form Template Method uygulanır, ilişkisiz sınıflardaysa metoda en uygun sınıf aranır. Long Method için Extract Method'un yanında Decompose Conditional (karmaşık koşulu ve dallarını adlandırılmış metotlara ayırmak), Replace Temp with Query (geçici değişkenleri sorgu metotlarına dönüştürmek) ve en zor durumlarda Replace Method with Method Object (metodu yeni bir sınıfa taşıyıp yerel değişkenleri üye alan yapmak) gösterildi. Long Parameter List için Introduce Parameter Object (`DateRange` gibi davranış da taşıyan bir parametre sınıfı) ve Preserve Whole Object (nesnenin kendisini göndermek) anlatıldı.

Dersin ortak teması, her eylemin kazanımlarıyla birlikte bedelinin de tartılmasıdır: metot sayısının artması, fazladan metot çağrısı ve işlem tekrarı, yeni sınıfların getirdiği bağlaşım, metot imzası değişince kaybolan geriye doğru uyumluluk. Hocaya göre bu bedeller çoğu zaman bakım kolaylığının yanında küçüktür; kuşkulu durumlarda profiler ile ölçüm yapılmalı, gerekirse sürüm denetimiyle geri dönülmelidir. Refactoring konusuna sonraki derste kalan kod kusurları ve eylemlerle devam edilecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.